

Parte 4. Conceptos

Desarrollo de Aplicaciones y Servicios Virtuales — Primer Examen Parcial

1. Diferencia entre Git y GitHub

Respuesta: Git es un **sistema de control de versiones**: un programa que se instala y se ejecuta localmente en la computadora, y que permite registrar el historial de cambios de un proyecto, crear ramas, comparar versiones y fusionar código. GitHub, en cambio, es una **plataforma en la nube** (un servicio web) que aloja repositorios de Git, agrega una interfaz gráfica y suma funciones de colaboración como Fork, Pull Request, Review, Issues y gestión de equipos.

Explicación: Git puede usarse sin GitHub, de forma completamente local, en cualquier proyecto. GitHub necesita de Git como base, pero le añade el componente social y remoto: un lugar centralizado donde varias personas pueden compartir, revisar y sincronizar su trabajo. Existen además otras plataformas equivalentes a GitHub (como GitLab o Bitbucket), lo que confirma que Git es la herramienta y GitHub es solo uno de los servicios que la utilizan.

2. Para qué sirve .gitignore

Respuesta: El archivo `.gitignore` le indica a Git qué archivos o carpetas **no debe rastrear ni incluir** en los commits, es decir, cuáles debe ignorar por completo.

Explicación: No todos los archivos de un proyecto deben subirse al repositorio: entornos virtuales, archivos temporales, cachés de compilación, credenciales o configuraciones específicas de cada máquina no aportan valor al historial compartido y pueden incluso representar un riesgo de seguridad. Al listar patrones (como `.venv/`, `__pycache__`, `*.pyc`, `.env`) dentro de `.gitignore`, Git deja de mostrar esos archivos como cambios pendientes y evita que se agreguen por accidente con comandos como `git add .`

3. Por qué .venv no debe almacenarse normalmente en GitHub

Respuesta: Porque `.venv` es un entorno virtual **generado localmente**, específico del sistema operativo y de la ruta de la computadora donde se creó, y puede reconstruirse fácilmente a partir de `requirements.txt`.

Explicación: Un entorno virtual contiene binarios y rutas absolutas que dependen de la máquina en la que fue creado; copiarlo a otra computadora normalmente no funciona correctamente. Además, suele pesar mucho y cambia constantemente, lo que ensuciaría el historial de commits sin aportar información útil. La práctica correcta es no versionar `.venv/` (excluyéndolo en `.gitignore`) y en su lugar compartir `requirements.txt`, de manera que cualquier persona pueda crear su propio entorno virtual e instalar exactamente las mismas dependencias con `pip install -r requirements.txt`.

4. Para qué sirve requirements.txt

Respuesta: `requirements.txt` es un archivo de texto que enumera las **bibliotecas (dependencias) de Python** que utiliza el proyecto, generalmente junto con la versión específica de cada una.

Explicación: Permite que cualquier persona que descargue o clone el proyecto pueda instalar exactamente las mismas dependencias con un solo comando (`pip install -r requirements.txt`), sin tener que adivinar qué bibliotecas se usaron ni en qué versión. Esto garantiza que el proyecto funcione de manera consistente en distintas computadoras y evita conflictos por diferencias entre versiones de las bibliotecas.

5. Diferencia entre Stage, Commit y Push

Respuesta:

- **Stage (`git add`)** — Selecciona qué cambios, de todos los que existen en la carpeta de trabajo, se incluirán en el próximo commit. Es un paso intermedio y reversible.
- **Commit (`git commit`)** — Guarda de forma permanente, en el **historial local**, los cambios que estaban en el área de preparación, junto con un mensaje descriptivo.

- **Push (`git push`)** — Envía los commits que existen en el repositorio **local** hacia el repositorio **remoto** (por ejemplo, GitHub), para publicarlos.

Explicación: Los tres representan niveles progresivos de “confirmación” de un cambio: primero se elige qué se quiere guardar (stage), después se guarda esa versión en el historial de la computadora (commit) y, finalmente, se comparte esa versión con los demás a través del repositorio remoto (push). Mientras un cambio solo ha sido “stageado” o “commitado”, únicamente existe en la máquina local; solo el push lo hace visible fuera de ella.

6. Por qué un repositorio puede tener varios commits antes de realizar un push

Respuesta: Porque el **commit es una operación local** y el **push es una operación de red** hacia el repositorio remoto; ambas son independientes entre sí y no ocurren automáticamente una tras otra.

Explicación: Git permite trabajar completamente sin conexión: se pueden crear tantos commits como se necesite mientras se avanza en el desarrollo (por ejemplo, uno por cada pequeña tarea o corrección), y todos ellos quedan guardados en el historial local. El push es una acción explícita que el desarrollador decide ejecutar cuando quiere publicar ese progreso acumulado en el repositorio remoto. Esto da flexibilidad: permite ordenar el trabajo en commits pequeños y descriptivos, y decidir en qué momento conviene compartir ese avance con el resto del equipo.